Job Scheduling Problem

A Backtracking / Constraint Satisfaction Approach to NP-Hard Scheduling

---

NAME: ADITHYA R

REGISTER NUMBER: 192311369

Topic: Job Scheduling (Backtracking / NP-Hard Problem)

Domain: Design and Analysis of Algorithms

1. Problem Definition

A company receives a set of n jobs, J = {J1, J2, ..., Jn}, that must be assigned to a set of m employees, E = {E1, E2, ..., Em}. Each job Ji is characterized by a deadline di (the latest time by which it must be completed), a processing time or duration ti, and a skill/resource requirement ri (the type of expertise or equipment the job needs). Each employee Ek has a set of skills Sk and an availability calendar Ak describing the time slots in which they are free to work.

The goal is to find an assignment A: J → E × T (mapping every job to an employee and a start time) such that every job is completed on or before its deadline, no employee is assigned two overlapping jobs, and every job is handled only by an employee qualified to perform it. If a feasible assignment exists, the algorithm must return it; otherwise it must correctly report infeasibility, or, in the optimization variant, maximize the number of jobs successfully scheduled.

This is a variant of the classical Job Sequencing with Deadlines / Job-Shop Scheduling problem. Because the search space of possible job-to-employee-to-time assignments grows combinatorially (in the worst case m^n possible mappings, further multiplied by ordering choices per employee), the general problem is NP-Hard. Exhaustive search is computationally infeasible for large n and m, which motivates a backtracking (systematic search with pruning) approach rather than brute-force enumeration.

2. Constraints

Three broad categories of constraints govern a feasible assignment. Each is defined precisely below, since the backtracking algorithm prunes the search tree by checking exactly these conditions at every partial assignment.

2.1 Time / Deadline Constraints

- Deadline feasibility: A job Ji assigned to employee Ek must finish, start(Ji) + ti ≤ di, i.e. it must complete on or before its deadline.
- Non-overlap in time: For any employee Ek, no two assigned jobs may occupy overlapping time intervals — an employee handles strictly one job at a time.
- Precedence (if applicable): Some jobs may depend on the completion of others, requiring start(Jj) ≥ finish(Ji) for a dependent pair (Ji, Jj).

2.2 Resource / Employee Availability Constraints

- Capacity constraint: Each employee has a fixed working window (e.g., shift hours); a job can only be placed inside that window.
- Single-assignment-at-a-time: Reinforces the time constraint — an employee's schedule is modeled as a list of disjoint intervals.
- Exclusivity of specialized resources: If a job requires a shared tool or machine in addition to an employee, that resource must also be free for the job's duration.

2.3 Job Requirement Constraints

- Skill / qualification match: Job Ji can only be assigned to employee Ek if the required skill ri ∈ Sk.
- Job atomicity: A job, once started, must be completed by the same employee without interruption (no preemption in the basic model).
- Priority weighting (optional, for optimization variant): High-priority or high-value jobs may be preferred when not all jobs can be scheduled.

3. Constraint Interaction and Prioritization

These constraints are not independent — they interact and jointly shrink the feasible region of the search space. A useful way to reason about them is to order the checks from “cheapest and most restrictive to test first” to “most expensive or least restrictive.” This ordering directly determines how effectively the backtracking search prunes infeasible branches early, which is the main driver of practical efficiency.

Recommended evaluation order when attempting to place a job at a candidate (employee, time-slot) pair:

1. Skill match (job requirement constraint) — checked first because it is a simple set-membership test (O(1) with a lookup table) and eliminates the majority of employee candidates instantly.
2. Availability / non-overlap (resource constraint) — checked next against the employee's current interval list; this is the constraint most frequently violated once skill-eligible employees remain.
3. Deadline feasibility (time constraint) — checked last among the hard constraints, since it depends on the specific time slot chosen, which is only meaningful after an eligible, available employee has been identified.
4. Precedence / dependency constraints — verified whenever the job under consideration has prerequisite jobs, ensuring causal ordering is respected.

The interaction is important: a job may satisfy the skill constraint and the deadline constraint individually, yet still be infeasible because the only qualified employee is already occupied during the required window (a resource × time conflict). Similarly, an employee may be free (resource constraint satisfied) but unqualified (skill constraint violated), or qualified and free but the remaining time before the deadline is shorter than the job's duration (time constraint violated). Because a single failed check invalidates the entire branch, ordering the cheapest and most discriminating tests first (skill, then availability, then deadline arithmetic) minimizes wasted computation — this is the core idea behind constraint propagation used to strengthen backtracking.

4. Backtracking / Constraint Satisfaction Solution

The problem is modeled as a Constraint Satisfaction Problem (CSP): variables are the jobs J1..Jn, the domain of each variable is the set of (employee, time-slot) pairs, and the constraints are exactly the time, resource, and job-requirement rules defined in Section 2. Backtracking performs a depth-first exploration of this assignment tree, assigning one job at a time and abandoning (“pruning”) any partial assignment that violates a constraint before it can propagate further.

4.1 Algorithm Design

- Order jobs by increasing deadline (Earliest Deadline First heuristic) before recursion begins — this is a standard ordering heuristic that tends to find feasible solutions faster and matches the greedy intuition behind deadline scheduling.
- At each recursive call, pick the next unassigned job and iterate over candidate employees.
- For each candidate, run the ordered feasibility check from Section 3 (skill → availability → deadline → precedence).
- If feasible, tentatively assign the job (mark the employee's time slot as occupied) and recurse on the remaining jobs.
- If the recursive call fails (returns no solution), undo the assignment ("backtrack": free the time slot) and try the next candidate employee/slot.
- If no candidate works for the current job, backtrack to the previous job and try its next alternative.

4.2 Pseudocode

```
function ASSIGN-JOBS(jobs, employees, index):
    if index == length(jobs):
        return TRUE                         // all jobs successfully scheduled

    job ← jobs[index]                        // jobs pre-sorted by deadline

    for each employee E in employees:
        for each candidate slot T in E.free_slots:

            // --- Pruning checks (ordered cheapest-first) ---
            if job.skill NOT IN E.skills:            continue   // skill mismatch
            if NOT E.isFree(T, job.duration):         continue   // overlap
            if T + job.duration > job.deadline:       continue   // deadline miss
            if NOT precedenceSatisfied(job, T):       continue   // dependency

            // --- Tentative assignment ---
            E.occupy(T, job.duration)
            job.assignedTo ← (E, T)

            if ASSIGN-JOBS(jobs, employees, index + 1):
                return TRUE                 // success propagates up the
recursion

            // --- Backtrack: undo and try next candidate ---
            E.release(T, job.duration)
            job.assignedTo ← NULL

    return FALSE                            // no valid assignment for this job
```

4.3 Pruning Strategies Applied

- Bound pruning: If the remaining time until a job's deadline is less than its duration, the branch is cut immediately — no need to check employees at all.

- Forward checking: Before recursing, the algorithm can pre-filter the domain of not-yet-assigned jobs whose skill or slot options shrink because of the current assignment, detecting dead ends earlier than plain backtracking would.
- Ordering heuristic (Earliest Deadline First / Most Constrained Variable): Scheduling tightly-constrained jobs (fewest qualified employees, earliest deadlines) first reduces branching factor at the top of the tree, where pruning has the largest downstream effect.
- Symmetry / redundancy pruning: If two employees have identical skills and identical free slots, only one needs to be tried at a given decision point — the other is guaranteed to yield an equivalent outcome.

4.4 Recursive Assignment — Worked Example

Consider 4 jobs and 3 employees:

Job	Duration	Deadline	Required Skill	Qualified Employees
J1	2 hrs	t = 4	Wiring	E1, E2
J2	3 hrs	t = 5	Welding	E2
J3	1 hr	t = 3	Wiring	E1, E2
J4	2 hrs	t = 6	Painting	E3

Jobs are first sorted by deadline: J3 (t=3) → J1 (t=4) → J2 (t=5) → J4 (t=6). The recursion proceeds as follows:

1. Assign J3 → E1 at [0,1]. Skill ✓, availability ✓, deadline ✓ (finishes at 1 ≤ 3). Recurse.
2. Assign J1 → E1 at [1,3]. Skill ✓, availability ✓ (E1 free after hour 1), deadline ✓ (finishes at 3 ≤ 4). Recurse.
3. Assign J2 → E2 at [0,3] (only qualified employee). Skill ✓, availability ✓, deadline ✓ (finishes at 3 ≤ 5). Recurse.
4. Assign J4 → E3 at [0,2]. Skill ✓, availability ✓, deadline ✓ (finishes at 2 ≤ 6). Recurse → index reaches end — solution complete, no backtracking was needed on this path.

If, instead, E1 had already been booked from hour 0–2 by an earlier commitment, the algorithm would fail the availability check for J3 → E1, prune that branch without recursing further, and immediately try J3 → E2 — illustrating how pruning avoids exploring the doomed sub-tree beneath an infeasible partial assignment.

5. Justification of Constraint Satisfaction

The proposed backtracking solution satisfies all three constraint categories systematically and efficiently:

- Time constraints are enforced at every recursive step via the explicit deadline check (T + duration ≤ deadline) and the non-overlap check on each employee's interval list, so no output of the algorithm can ever contain a late or double-booked job.

- Resource constraints are enforced by only considering (employee, slot) pairs drawn from that employee's currently free intervals, which are updated (occupied/released) on every assignment and backtrack — guaranteeing the employee's calendar always reflects the true state of the search.
- Job requirement constraints (skill matching) are enforced as the very first check in the pruning order, so a job is never even tentatively placed with an unqualified employee.

Correctness follows from exhaustiveness: backtracking systematically explores every branch of the assignment tree and only commits to a branch once all constraints for that step pass, and it undoes any branch that later proves infeasible. Because the algorithm never accepts a partial assignment that violates a constraint, and because it backtracks fully on failure, it explores the entire reachable feasible space and is guaranteed to find a valid schedule whenever one exists (completeness), or correctly report failure otherwise (soundness).

Efficiency is achieved not by reducing the worst-case complexity of the NP-Hard problem itself (which remains exponential, O(m^n) in the naive case), but by aggressively pruning branches early using cheap, highly discriminating tests (skill match, deadline bound) before expensive ones, combined with the Earliest-Deadline-First ordering heuristic that tends to expose infeasibility — or find a solution — much earlier in the search than an unordered brute-force enumeration would. In practice, for realistic job counts, this reduces the explored search space by several orders of magnitude compared to exhaustive assignment enumeration.

6. Conclusion

The job scheduling problem is a natural fit for a backtracking/CSP formulation: jobs are variables, (employee, time) pairs are domains, and the time, resource, and skill constraints are the CSP's constraint set. By carefully prioritizing constraint checks — cheap and discriminating tests before expensive ones — and applying pruning strategies such as bound checks, forward checking, and deadline-based ordering, the recursive assignment procedure finds a feasible (or provably optimal, if extended with an objective function) schedule while avoiding the full exponential blow-up of brute-force search. This approach generalizes directly to real-world workforce scheduling systems, where additional constraints (multi-skill jobs, shift patterns, priority weighting) can be folded into the same pruning framework without changing the overall algorithmic structure.